Homework 2 Report

1. Problem 1:

Unlike the RGB model, which represents color using the primary light components (Red, Green, and Blue), YCbCr separates image data into Luminance (Y), representing brightness and Chrominance (Cb and Cr), representing color differences. This separation is beneficial because the human eye is more sensitive to changes in brightness than in color, allowing for more efficient processing, storage, and compression of image data.

The conversion between RGB and YCbCr is defined by a linear transformation consisting of a matrix multiplication followed by an offset vector.

1.1 Algorithms

RGB to YCbCr Conversion

To convert an image from RGB to YCbCr, the input RGB values (typically normalized or treated as double-precision) are multiplied by a specific transformation matrix. The RGB values are first normalized to the range [0,1] before applying the transformation matrix. An offset vector [16, 128, 128] is then added to shift the resulting values into the standard digital range, where 16 represents the minimum luminance and 128 is the neutral point for color differences.

The mathematical model is:

$Y = 16 + (65.481 \times R) + (128.553 \times G) + (24.966 \times B)$
$Cb = 128 + (-37.797 \times R) + (-74.203 \times G) + (112.000 \times B)$
$Cr = 128 + (112.000 \times R) + (-93.786 \times G) + (-18.214 \times B)$

Algorithm:

1. Convert the input RGB image to double precision and normalize the RGB values to the range [0,1].
2. Apply the transformation matrix to each pixel's R, G, and B components.
3. Add the offset vector [16, 128, 128].
4. Round the resulting values and cast them back to an 8-bit integer format (uint8) for display and storage.

YCbCr to RGB Conversion

The reverse process requires finding the inverse of the transformation matrix used in the first step. By subtracting the offset and multiplying by the inverse matrix, we can recover the original Red, Green, and Blue components.

The mathematical model is:
[R G B]^T = T^-1 * [Y-16, Cb-128, Cr-128]^T

Algorithm:

1. Subtract the digital offset [16, 128, 128] from the Y, Cb, and Cr channels.
2. Multiply the adjusted vector by the inverse of the transformation matrix.
3. Rescale the resulting values to the [0, 255] range.
4. Apply rounding and clip values to ensure they remain within the valid 8-bit range (0 to 255) before converting back to uint8.

Luminance Reduction Function
This function demonstrates the YCbCr space by manipulating brightness without intentionally altering the color data.

Algorithm:

1. Convert the source RGB image to the YCbCr color space.
2. Isolate the Y channel (Luminance) and multiply all values by 0.5 (reducing brightness by 50%).
3. Leave the Cb and Cr channels unchanged to preserve the original color information.
4. Convert the modified YCbCr image back to the RGB color space for viewing.

1.2 Result
The implementation was tested using the images Everest_expedition.jpg and Everest_kalapatthar.jpg.

For Everest_expedition.jpg, the maximum absolute difference between the custom and MATLAB functions was 1, and for Everest_kalapatthar.jpg it was 2, confirming that the implementation closely matches MATLAB's built-in functions.

When comparing the custom implementation against the MATLAB toolbox functions (rgb2ycbcr and ycbcr2rgb), the results were nearly identical, with only minor differences caused by rounding floating-point numbers.



Figure 1: Original and reduced-luminance RGB images of Everest_expedition.jpg

Effect of Reducing Luminance

In the provided figure (Figure 1), reducing the Y component by half has a profound effect on the RGB image:

- **Visual Dimming:** The "Reduced Luminance" image appears significantly darker than the original. This is because the Y channel directly controls the perceived intensity of light in the scene.
- **Color Preservation:** Despite the image being darker, the hues remain consistent. The yellow of the tents and the blue of the sky still look yellow and blue; they simply appear as if photographed under much lower lighting conditions (e.g., at twilight).
- **RGB Impact:** In the RGB space, this operation manifests as a proportional decrease across all three channels (R, G, and B) for every pixel. Because the Y component is a weighted sum of RGB and contributes significantly to the reconstruction of R, G, and B values, reducing Y decreases the overall intensity of all three RGB channels while preserving the relative color balance.

2. Problem 2:

An 8-bit grayscale digital image can be represented as eight bit planes, ranging from the Least Significant Bit (LSB, bit 0) to the Most Significant Bit (MSB, bit 7). Each bit plane contributes differently to the pixel intensity values, the histogram, and ultimately the visual appearance of the image. This problem investigates the effect of zeroing lower-order or higher-order bit planes on both the histogram and the visual characteristics of an image. The verification is performed on LENNA.JPG.

2.1 Theoretical Analysis

Setting Lower-Order Bit Planes to Zero

1. **Effect on Histogram:** Zeroing lower-order bits acts as a quantization process. For example, setting the lower 3 bits (bit 0–2) to zero restricts all pixel values to

multiples of 8. The histogram shows discrete spikes separated by intervals of 8, creating a "comb-like" pattern.

2. **Why:** The lower bits encode fine-grained intensity variations. Removing them eliminates subtle differences between adjacent gray levels.
3. **Impact on Appearance:** The image remains visually very similar. Structural details and edges are preserved, while smooth gradients may appear slightly "posterized" or stepped.

Setting Higher-Order Bit Planes to Zero

1. **Effect on Histogram:** The entire histogram is crushed to the far left (the dark end). All pixel values are reduced to a very small range (e.g., 0–7 if the top 5 bits are removed).
2. **Why:** The higher-order bits contain the majority of the numerical value of a pixel. Removing them discards the primary luminance data.
3. **Impact on Appearance:** The image becomes nearly black or appears as random noise. The structural information, such as edges and shapes, is lost because the data that defines contrast has been deleted.

2.2 Experimental Verification (LENNA.JPG)

A MATLAB program was implemented to verify the theoretical arguments regarding bit-plane manipulation using LENNA.JPG. To isolate the effects of different bit planes, the lower three bits were removed using a bitwise AND operation with a mask value of 248 (binary 11111000), while the higher bits were removed using a mask value of 7 (binary 00000111).

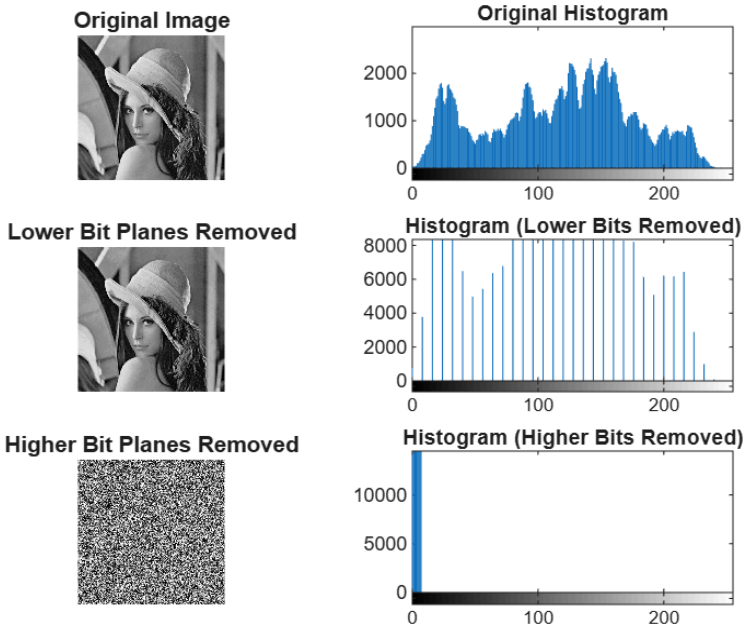


Figure 2: Comparison of the original image with images where lower and higher bit planes are removed, along with their corresponding histograms showing the resulting intensity distribution changes.

Experimental Results from Figure 2

- **Original Image:**
  The original LENNA.JPG displays a clear portrait with smooth grayscale gradients. Its histogram shows a continuous distribution of pixel intensities spanning the full range from 0 to 255.

- **Lower Bit Planes Removed:**
  As shown in Figure 2, the resulting image appears visually almost identical to the original at first glance. However, the histogram reveals a distinct comb-like pattern, where pixel intensities occur only at discrete intervals. These spikes occur because removing the lower three bits forces all pixel values to become multiples of 8.

- **Higher Bit Planes Removed:**
  The effect of removing the higher bit planes is much more drastic. The image structure is completely destroyed and appears as random noise, making the subject unrecognizable. The histogram shows that all pixel intensity values are compressed into the narrow range [0, 7], clustered near the left side of the histogram.

2.3 Result

The experimental results support the theoretical argument regarding the importance of different bit planes. When the lower-order bit planes were removed, the image retained its main structure and recognizable features. However, the histogram displayed discrete spacing between values, confirming that these lower bits primarily represent fine intensity precision rather than essential structural information.

In contrast, removing the higher-order bit planes destroyed the recognizable structure of the image and shifted the histogram almost entirely toward zero intensity values. This confirms that the higher bit planes contain the most significant information for luminance, contrast, and structural perception within the image.

3. Problem 3: RGB Histogram Matching

Histogram matching (also called histogram specification) is an image processing technique that adjusts the intensity distribution of one image so that it matches the histogram of another reference image. Unlike histogram equalization, which spreads intensities uniformly, histogram

matching transforms the source image so that its cumulative distribution function (CDF) resembles that of a target image.

For color images, histogram matching can be applied independently to the Red, Green, and Blue (RGB) channels. This allows the color tone and contrast characteristics of a target image to be transferred to the source image while preserving the original spatial structure.
In this problem, a custom MATLAB function was implemented to perform RGB histogram matching without using any predefined histogram functions. The function was tested using Atlas-Mercury.png as the source image and techno-trousers.png as the target image

3.1 Algorithms

Histogram matching is performed independently for the **Red, Green, and Blue channels** of the image. The procedure consists of the following steps.

1. **Intensity Frequency Counting:**
   For each color channel in both the source and target images, every pixel is examined and the occurrences of each intensity level from 0 to 255 are counted to construct the histogram.
2. **Probability Density Function (PDF):**
   The frequency counts are normalized by dividing by the total number of pixels in the image. This produces the probability distribution of pixel intensities.
3. **Cumulative Distribution Function (CDF):**
   The cumulative distribution function is obtained by computing the running sum of the PDF values. The CDF represents the cumulative probability of intensity values and is used to determine the mapping between source and target intensities.
4. **Mapping Table Construction:**
   For every intensity level i in the source image, the algorithm identifies an intensity level j in the target image that minimizes the difference between the two CDF values:
   $|CDF_{source}(i) - CDF_{target}(j)|$
   This establishes a mapping from the source intensity to the corresponding target intensity.
5. **Intensity Re-mapping:**
   The source image pixels are replaced using the mapping table generated in the previous step. This produces the final RGB histogram-matched output image.

3.2 Experimental Results

The algorithm was tested using:

- **Source image:** Atlas-Mercury.png
- **Target image:** techno-trousers.png

The resulting output image preserves the spatial structure of the source image while adopting the color and contrast characteristics of the target image. Figure 3 shows the source image, the target image, and the histogram-matched output.



Figure 3: Source image (Atlas-Mercury), target image (techno-trousers), and the resulting histogram-matched image.

From visual observation, the matched output image clearly adopts the warm reddish color tones and lighting characteristics present in the target image while maintaining the structural features of the rocket scene from the source image.

3.3 Scientific Verification

The correctness of the histogram matching algorithm was verified by comparing the cumulative distribution functions (CDFs) of the matched output and the target image.

If the algorithm is implemented correctly, the CDF of the histogram-matched image should closely follow the CDF of the target image.

To validate this, the CDFs of the Red channel were plotted for both the target image and the matched output.

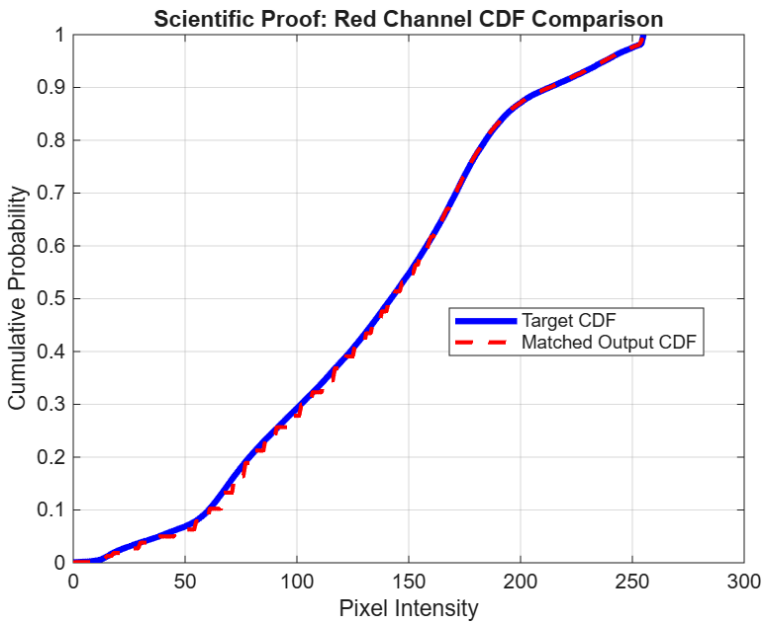


Figure 4: Comparison of cumulative distribution functions (CDFs) for the Red channel of the target image and the matched output image.

As shown in Figure 4, the CDF of the matched output (dashed red curve) closely overlaps with the CDF of the target image (solid blue curve). This close alignment confirms that the intensity distribution of the source image has been successfully transformed to match the target distribution.

3.4 Conclusion

The implemented RGB histogram matching algorithm successfully adjusts the intensity distribution of a source image to resemble that of a target image. The resulting image adopts the color tone and contrast characteristics of the target while preserving the structural content of the source image.

The scientific verification using cumulative distribution functions confirms that the matched output closely follows the target intensity distribution, demonstrating that the histogram matching algorithm has been correctly implemented.

4. Problem 4:

Spatial filters are often applied sequentially to achieve different objectives. An averaging filter is commonly used to reduce noise by smoothing the image, while a Laplacian filter is used to enhance edges and small details. This problem investigates whether applying these two filters in different orders produces the same result.

4.1 Would the result be the same if the order of these operations were reversed?
Both the averaging mask and the Laplacian mask are linear spatial filters applied through convolution. A fundamental property of convolution is commutativity, which states that the order of convolution operations does not affect the final result.

Mathematically:
$g(x,y) = f(x,y) * h_{avg}(x,y) * h_{lap}(x,y)$

Because convolution is commutative:
$f * h_{avg} * h_{lap} = f * h_{lap} * h_{avg}$

Therefore, applying the averaging filter followed by the Laplacian filter should theoretically produce the same result as applying the Laplacian filter followed by the averaging filter. This means the order of the filters does not change the theoretical output when both operations are linear convolutions.

4.2 Experimental Verification (LENNA.JPG)
A MATLAB experiment was conducted using **LENNA.JPG** to verify this argument. Two filtering sequences were applied:

1. Averaging filter followed by Laplacian filtering
2. Laplacian filtering followed by averaging filtering

The results of both operations are shown in **Figure 5**.

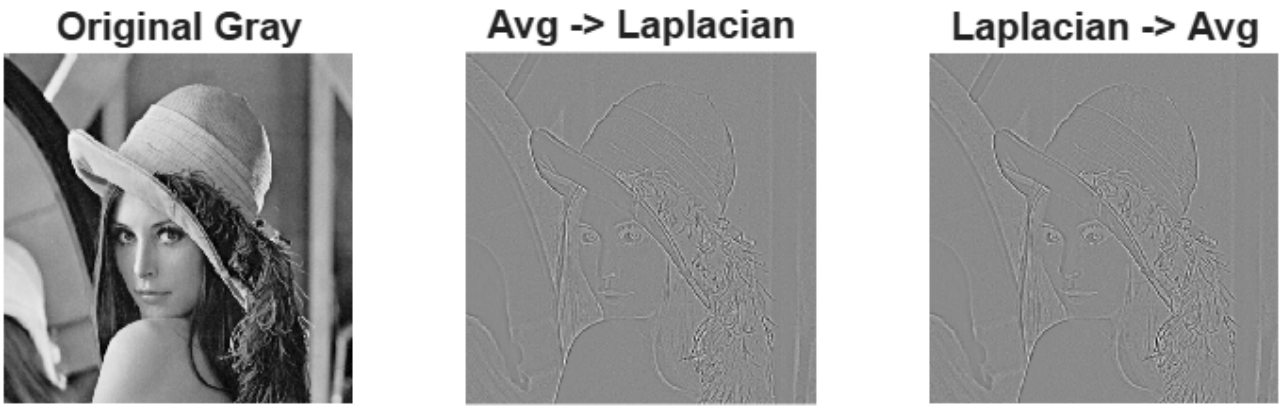


Figure 5: Comparison of filtering order using LENNA.JPG. From left to right: original grayscale image, result of applying averaging followed by Laplacian filtering, and result of applying Laplacian followed by averaging filtering.

4.3 Result Analysis

The experimental results show that the two filtered images appear **very similar**, confirming the theoretical expectation that the order of convolution operations should not change the result.

Any small differences observed between the two outputs are not due to the filter order itself but are caused by **implementation details**, such as:

- boundary padding used during convolution
- numerical rounding in floating-point computations

These practical factors introduce minor variations near the image boundaries.

4.4 Conclusion

Theoretically, applying an averaging filter followed by a Laplacian filter produces the same result as applying the Laplacian filter first and then the averaging filter. This is due to the commutative property of convolution for linear spatial filters.

The MATLAB experiment supports this theory, as the two outputs are nearly identical. Any slight differences arise from implementation factors such as padding and numerical precision rather than the filter order itself.

5. Resources that Helped Me

MATLAB Documentation:

MATLAB Logical Bitwise Operations
https://www.mathworks.com/help/matlab/ref/bitand.html
MATLAB Image Histograms
https://www.mathworks.com/help/images/ref/imhist.html
MATLAB Spatial Filtering
https://www.mathworks.com/help/images/ref/imfilter.html
MATLAB Predefined Filter Design
https://www.mathworks.com/help/images/ref/fspecial.html

Others:

YCbCr Color Space Standards
https://www.youtube.com/watch?v=3dET-EoIMM8
Histogram Matching Logic
https://paulbourke.net/miscellaneous/equalisation/
Histogram (frequency distribution concept)
https://asq.org/quality-resources/histogram

Spatial Filtering Concepts
https://www.ni.com/docs/en-US/bundle/ni-vision/page/spatial-filtering.html

Course Material:

Course lecture notes